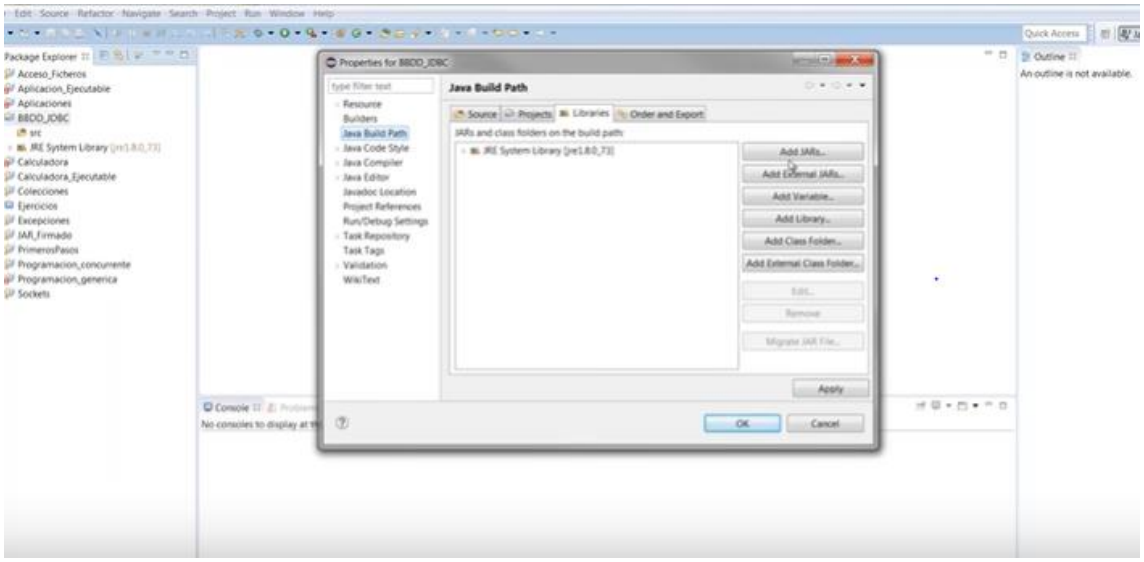
JDBC

Es necesario tener:

1- Mysql
2- Driver jdbc mysql
3- Agregar el driver al class path

Una vez que se ha descargado el driver, lo que vamos a hacer es ir al proyecto en eclipse (click derecho) → propiedades → java build path

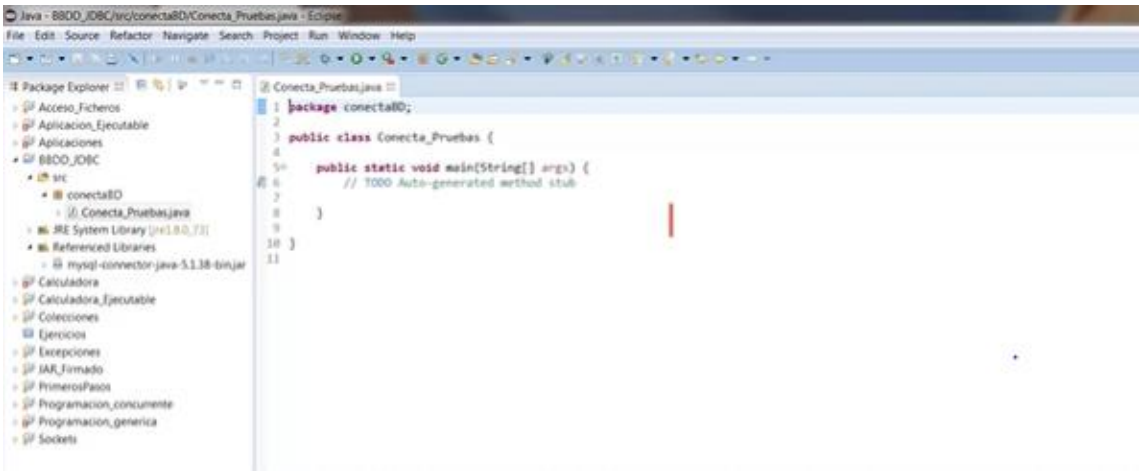


Pulsar el botón "add external jar"

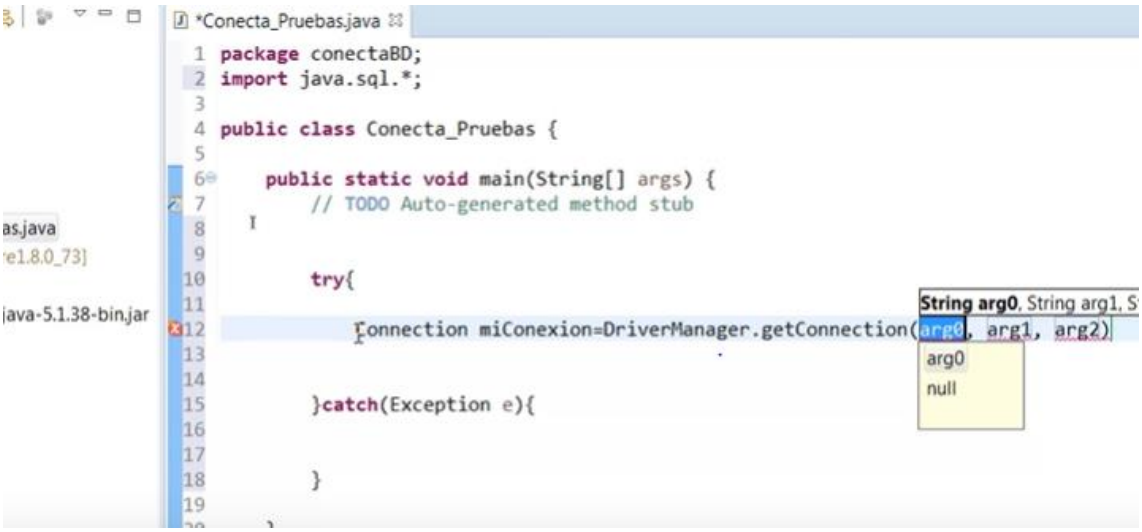
- Se agregan en el proyecto las librerias correspondientes

<u>Pasos</u>:

1. Con xamp o wamp lo arrancas, el phpmyadmin, a través de la cual puedo manejar mi base de datos.
2. Creamos en nuestro proyecto una clase, después de crear un paquete por no trabajar con el de por defecto. A la clase la llamo Conecta_prueba



3. Lo primero es crear la conexión y como lanza una excepción, debemos capturarla con un try catch. Necesitamos crear un objeto de tipo conexión.
   Necesito usar la clase driverMAnager que tiene un método getConexion, (lo vemos en la api)
4. Nos pide un usuario y una contraseña, el usuario por defecto en una base de datos mysql si no se ha cambiado es: user: root , pssw: “”
5. Vemos que pertenece al paquete, java.sql debemos importarlo.
   Import.java.sql.*
6. Realizamos la conexión:



7. Si no se ha modificado, por defecto el puerto (que abre la base de datos mysql con ella) es el 3306, por tanto, la ruta y el nombre de la bbdd es:

```
ecta_Pruebas.java
ackage conectaBD;
mport java.sql.*;

ublic class Conecta_Pruebas {

   public static void main(String[] args) {
       // TODO Auto-generated method stub


       try{

           Connection miConexion=DriverManager.getConnection("jdbc:mysql://localhost:3306/pruebas", a


       }catch(Exception e){


       }

   }
```

8. Para ver si es ese puerto:
Ir a phpmyadmin y en la consola de SQL (pestaña), poner lo siguiente:

Pulsamos el botón continuar y tenemos el dato.

```
n=DriverManager.getConnection("jdbc:mysql://localhost:3306/pruebas", "root", "")
```

8. El siguiente paso es crear un objeto statement, para después poder ejecutar una consulta sql
9. Para esto sería lo siguiente:

```
public static void main(String[] args) {
    // TODO Auto-generated method stub

    try{
    //1. CREAR CONEXIÓN
        Connection miConexion=DriverManager.getConnection("jdbc:mysql://localhc
    //2. CREAR OBJETO STATEMENT
        Statement miStatement =miConexion.createStatement();

    }catch(Exception e){

    }
```

10. Siguiente paso, ejecutar la instrucción SQL. Necesitaremos el objeto de tipo Statement
    Se usa el método executeQuery
    Nos crearemos un objeto de tipo ResultSet y ahí guardaremos el resultado de la consulta.

```
try{
//1. CREAR CONEXIÓN
    Connection miConexion=DriverManager.getConnection("jdbc:mysql://localhost:3306/prueb
//2. CREAR OBJETO STATEMENT
    Statement miStatement =miConexion.createStatement();
//3. EJECUTAR SQL
    ResultSet miResultset=miStatement.executeQuery("SELECT * FROM PRODUCTOS");
```

11. resultSet es un objeto en nombre de tabla, asiq ahora habrá que recorrerlo para así poder ver los datos.
    Hay métodos first (se pone el cursor en el primer registro), next (mueve el cursor hacia delante…)
    Usamos, por ejemplo, bucle while y recorremos el objeto, con la condición mientras haya un siguiente registro.

```
15
16          //2. CREAR OBJETO STATEMENT
17
18              Statement miStatement =miConexion.createStatement();
19
20          //3. EJECUTAR SQL
21
22              ResultSet miResultset=miStatement.executeQuery("SELECT * FROM PRODUCTOS");
23
24          // 4. RECORRER EL RESULTSET
25
26              while(miResultset.next()){
27
28
29
30              }
31
32
33          }catch(Exception e){
```

Console | Problems | Javadoc | Declaration | Search

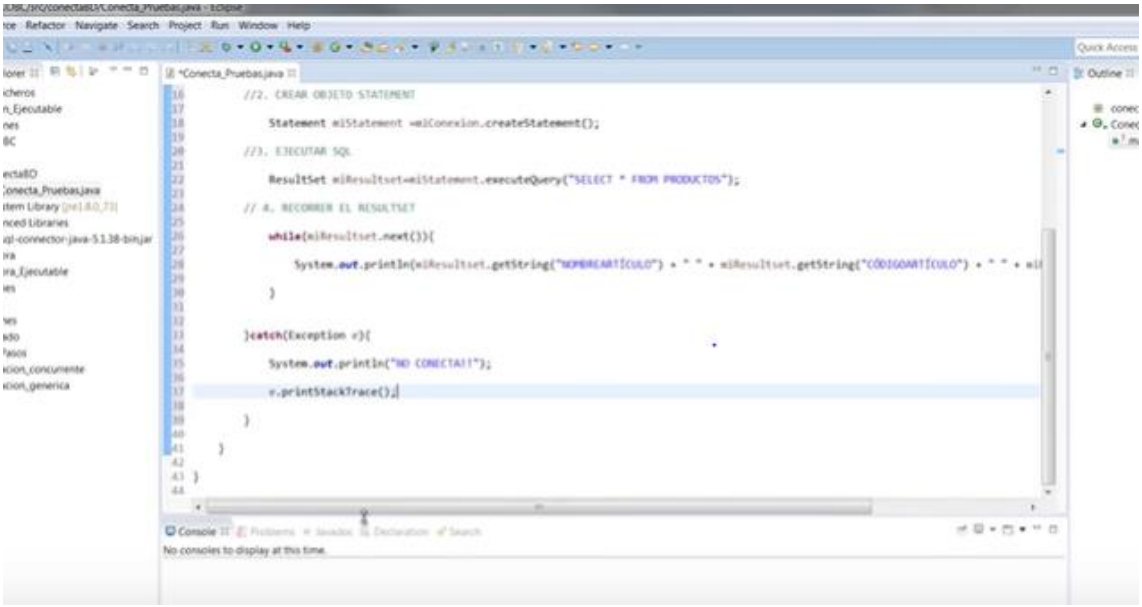
12. Imprimo los campos que queramos, usando el método getString de la interface resultSet, pasándole el nombre tal cual de la tabla y concateno para que me devuelva la siguiente columna, no sólo una.

```
*Conecta_Pruebas.java
try{
    //1. CREAR CONEXIÓN
        Connection miConexion=DriverManager.getConnection("jdbc:mysql://localhost:3306/pruebas", "root", "");
    //2. CREAR OBJETO STATEMENT
        Statement miStatement =miConexion.createStatement();
    //3. EJECUTAR SQL
        ResultSet miResultset=miStatement.executeQuery("SELECT * FROM PRODUCTOS");
    // 4. RECORRER EL RESULTSET
        while(miResultset.next()){
            System.out.println(miResultset.getString("NOMBREARTÍCULO") + " " + miResultset.getString("CÓDIGOARTÍCULO") + " " + mi
        }
    }catch(Exception e){
```

Console | Problems | Javadoc | Declaration | Search

No consoles to display at this time.

13. Debemos rellenar el catch, es decir que nos imprima un mensaje si hay un error:

```
va
.0_73]
5.1.38-bin.jar
23
24          // 4. RECORRER EL RESULTSET
25
26              while(miResultset.next()){
27
28                  System.out.println(miResultset.getString("NOMBREARTÍCULO") + " " +
29
30              }
31
32
33          }catch(Exception e){
34
35              System.out.println("NO CONECTA!!");
36
37          }
38
39      }
40
```

13. Si queremos que nos dé información adicional de por qué no conecta es útil imprimir la pila, para ellos usamos el objeto del evento: